



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Earliness-Tardiness Scheduling Problems, II: Deviation of Completion Times About a Restrictive Common Due Date

Nicholas G. Hall, Wieslaw Kubiak, Suresh P. Sethi,

To cite this article:

Nicholas G. Hall, Wieslaw Kubiak, Suresh P. Sethi, (1991) Earliness-Tardiness Scheduling Problems, II: Deviation of Completion Times About a Restrictive Common Due Date. Operations Research 39(5):847-856. <https://doi.org/10.1287/opre.39.5.847>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1991 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

EARLINESS-TARDINESS SCHEDULING PROBLEMS, II: DEVIATION OF COMPLETION TIMES ABOUT A RESTRICTIVE COMMON DUE DATE

NICHOLAS G. HALL

The Ohio State University, Columbus, Ohio

WIESLAW KUBIAK

Memorial University of Newfoundland, St. John's, Newfoundland, Canada

SURESH P. SETHI

University of Toronto, Toronto, Ontario, Canada

(Received April 1989; revisions received September 1989, January 1990; accepted April 1990)

A companion paper (Part I) considers the problem of minimizing the weighted earliness and tardiness of jobs scheduled on a single machine around a common due date, d , which is unrestrictively late. This paper (Part II) considers the problem of minimizing the unweighted earliness and tardiness of jobs, allowing the possibility that d is early enough to constrain the scheduling decision. We describe several optimality conditions. The recognition version of the problem is shown to be NP-complete in the ordinary sense, confirming a well known conjecture. Moreover, this complexity definition is precise, since we describe a dynamic programming algorithm which runs in pseudopolynomial time. This algorithm is also extremely efficient computationally, providing an improvement over earlier procedures, of almost two orders of magnitude in the size of instance that can be solved. Finally, we describe a special case of the problem which is polynomially solvable.

We will consider the following single machine scheduling environment. There are n jobs with known integer processing times $p_1, \dots, p_n$ and a known integer due date $d \geq 0$, at which time we would, ideally, complete each job. A cost is incurred whenever a job is completed either earlier than, or later than, d . A summary of related literature appears in the companion paper by Hall and Posner (1991), referred to as Part I, where d is assumed to be late enough not to influence the scheduling decision. The present paper, however, eschews the simplifying assumption that the due date is unrestrictively late. Indeed, the due date will frequently be an important factor in our scheduling decision. In this environment, the minimization of total (unweighted) earliness and tardiness is attempted.

There have been at least two earlier studies of this more general problem. Bagchi, Sullivan and Chang (1986) describe an implicit enumeration procedure which is effective at solving problems up to 15 jobs in size. The derivation of optimality conditions and lower bounds by Szwarc (1989) permits the design of

a branch-and-bound algorithm capable of solving larger problems, up to $n = 25$. Doing so, however, may involve the examination of several thousand subproblems.

This paper contains a description of several optimality conditions for, and other characteristics of, the problem. We prove that the recognition question which corresponds to the minimization of total earliness and tardiness is NP-complete in the ordinary sense. This result confirms the conjecture of Baker and Scudder (1990), and lends support to the enumerative approaches of Bagchi, Sullivan and Chang (1986) and Szwarc. We then describe a pseudopolynomial time algorithm, thereby demonstrating that the problem is not NP-complete in the strong sense. This algorithm, based on dynamic programming, is capable of solving large instances of the problem, up to 1,000 jobs, in under three minutes of CPU time on a minicomputer. The algorithm can also be extended to solve an important class of nonlinear cost scheduling problems (Vani and Raghavachari 1987, Bagchi, Sullivan and Chang 1987) that generalizes the total

Subject classification: Production/scheduling: deterministic, single machine sequencing.

earliness and tardiness problem (see Remarks 1 and 3). For a special case of the problem that we identify, an efficient algorithm can be provided.

1. OPTIMALITY CONDITIONS

An instance of the constrained unweighted earliness and tardiness problem CET is defined by n jobs with arbitrary, known integer processing times $p_1, \dots, p_n$ and a common due date, d . Throughout this paper, we assume the jobs to be ordered so that $p_1 \leq \dots \leq p_n$. Let $C_j(\sigma)$, written as C_j , denote the completion time of job j in schedule σ for $j = 1, \dots, n$, and $z(\sigma)$ denote the cost of σ . The objective is to minimize the sum of the earliness and tardiness of the jobs, that is:

$$\min_{\sigma \in \Pi} \left[z(\sigma) = \sum_{j=1}^n \max\{d - C_j, 0\} + \sum_{j=1}^n \max\{C_j - d, 0\} = \sum_{j=1}^n |C_j - d| \right]$$

subject to $C_j \geq p_j, j = 1, \dots, n$

where Π denotes the set of all possible nonpreemptive, single machine schedules. The requirement that $C_j \geq p_j, j = 1, \dots, n$, ensures that no job can begin processing before time zero. If $d \geq \sum_{j=1}^n p_j$, we say that the problem is unconstrained, and a polynomial solution procedure exists (Kanet 1981). In general, however, this will not be the case.

We now describe several conditions which are necessary for a schedule to be optimal for CET. We also identify several properties that would potentially simplify the search for an optimal solution, but which, as shown by example, do not hold in every problem instance. It is clear that any schedule with idle time between jobs can have its cost reduced by the removal of that idle time. Thus, we only consider schedules that process jobs consecutively. Define $E = \{1 \leq j \leq n \mid C_j < d\}$, $E' = \{1 \leq j \leq n \mid C_j \leq d\}$, $T = \{1 \leq j \leq n \mid C_j - p_j \geq d\}$ and $T' = \{1 \leq j \leq n \mid C_j > d\}$. We will describe an instance of CET using the notation $[n; p_1, p_2, \dots, p_n; d]$. Thus, for example, $[2; 3, 5; 4]$ defines an instance with two jobs, $p_1 = 3, p_2 = 5$ and $d = 4$.

A solution to an instance of CET is described by a sequence (ordering) of the jobs $1, \dots, n$, and a specification of the completion time of the unique job, α , which starts processing before d and completes at, or later than, d . Since processing is consecutive, this is sufficient to describe the schedule. For example, $(1, 2@, 3)$ means that we process jobs in the sequence 1, 2, 3, and job 2 completes exactly at d . However, $(1, 2@10, 3)$ means that while the sequence

is as before, job 2 starts before d and finishes after d , at time 10. With α defined in this way, $C_\alpha = d$ if a job finishes at d , and $C_\alpha > d$ if no job does so. It is easy to see that, if $d = 0$, then a solution is specified by an SPT sequence of the jobs. We begin with several results which, taken together, suggest that an optimal schedule for CET may be difficult to characterize.

First, there exist instances of CET for which no optimal schedule completes a job exactly at d , as shown by the instance $[1; 10; 6]$ with a unique optimal schedule of $(1@10)$. This is not true for the unconstrained version of the problem studied by Kanet. Second, there exist instances of CET for which every optimal schedule completes a job exactly at d , as shown by the instance $[3; 5, 10, 20; 10]$ with a unique optimal schedule of $(2@, 1, 3)$. If we let t^* denote the starting time of the first job processed in an optimal schedule, then Szwarc shows that $t^* = 0$ in some instances and $t^* > 0$ in others. To summarize these preliminary results, we do not know whether a job completes at d , nor do we know whether processing starts at 0. However, these results are not independent of each other, as we now demonstrate.

Property 1. *For every instance of CET, there exists at least one of the following:*

- an optimal schedule with $t^* = 0$*
- an optimal schedule with $C_\alpha = d$.*

Proof. Assume that $t^* > 0$ in an optimal solution. If a job completes at d , the proof is complete. Assuming that no job completes at d , let $0 < \epsilon < \min\{d - (C_\alpha - p_\alpha), C_\alpha - d\}$, and Δ_E (respectively, Δ_T) denote the change in cost from starting all jobs ϵ units of time earlier (respectively, later). Clearly, $\Delta_E = -\Delta_T$, therefore $\min\{\Delta_E, \Delta_T\} \leq 0$, and since by assumption the solution is optimal, $\min\{\Delta_E, \Delta_T\} \geq 0$, thus $\Delta_E = \Delta_T = 0$. It is easy to see that all jobs can be moved earlier until either $t^* = 0$ or $C_\alpha = d$.

Property 1 greatly simplifies the solution of problem instances for which $t^* > 0$, because we need consider only schedules in which $C_\alpha = d$. Given E' and T , Property 2 states that an optimal schedule can easily be found.

Property 2. (Bagchi, Sullivan and Chang 1986) *For every instance of CET, in each optimal schedule the jobs in E' are ordered by nonincreasing processing times, and the jobs in T are ordered by nondecreasing processing times.*

Proof. The proof follows from a job interchange argument.

Thus, many of the jobs have the V-shaped schedule that frequently exists in nonregular measure scheduling problems (see Eilon and Chowdhury 1977, or Kanet, for example). In Part I, this property holds for all the jobs. However, we note that the definitions of E' and T exclude α if $C_\alpha > d$. Let $e_{\min} = \min\{j \in E'\}$, $t_{\min} = \min\{j \in T\}$, where $e_{\min} = +\infty$ if $E' = \emptyset$ and $t_{\min} = +\infty$ if $T = \emptyset$. Thus $e_{\min}$ is the index of the last job completed by time d , and $t_{\min}$ is the index of the first job started at or after d . We need two further definitions before examining the relative values of p_α , $p_{e_{\min}}$ and $p_{t_{\min}}$.

Definition 1. A schedule is **strongly V-shaped** if $p_\alpha \leq \min\{p_{e_{\min}}, p_{t_{\min}}\}$.

It is tempting to conjecture that, for every instance of CET, there exists an optimal solution having the property of being strongly V-shaped. Such a result would be extremely useful, because it would imply that only the shortest job could start before d and finish after d . Unfortunately, the conjecture is not correct.

Fact 1. There exist instances of CET for which no optimal solution is strongly V-shaped.

Proof. By example. Consider the instance [2; 1, 11; 10]. The unique optimal solution is (2@11, 1), with a cost of 3, thus $p_\alpha = 11 > 1 = p_{t_{\min}}$.

Definition 2. A schedule is **weakly V-shaped** if $p_\alpha \leq \max\{p_{e_{\min}}, p_{t_{\min}}\}$.

Note that a weakly V-shaped schedule with $C_\alpha = d$ is a V-shaped schedule in the traditional sense, i.e., with respect to the due date, and is also strongly V-shaped. While the strongly V-shaped property does not generally hold, the weakly V-shaped property will be very useful in the design of our dynamic programming algorithm. For the special case of $t^* = 0$, a similar property has been proved by Bagchi, Sullivan and Chang (1986).

Property 3. For every instance of CET, each optimal schedule has the weakly V-shaped property.

Proof. (A similar result has been proved by Raghavachari (1986). Our proof that follows uses a different organization of subcases, which permits easier use of Property 3 (see Remark 1, Theorem 2 and Theorem 3).) Let E and T define an optimal schedule in which, contrary to Property 3, $p_\alpha > \max\{p_{e_{\min}}, p_{t_{\min}}\}$. The following three cases are shown in Figure 1.

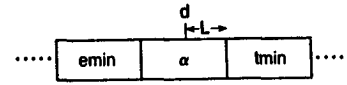


Figure 1. Schedule for Property 3.

Case 1. $C_\alpha = d$. Then $\alpha \in E'$ and, from Property 2, $p_\alpha \leq p_{e_{\min}}$ in each optimal schedule, a contradiction.

Case 2. $C_\alpha = d + L$, where $p_\alpha \geq 2L$.

Case 3. $C_\alpha = d + L$, where $p_\alpha < 2L$.

The significance of the relative values of p_α and $2L$ is that if $p_\alpha \geq 2L$ we will interchange jobs α and $e_{\min}$, but if $p_\alpha < 2L$ we will interchange jobs α and $t_{\min}$.

Let C'_j denote the completion time of job j after an interchange of jobs has taken place. For Case 2, let Δ denote the change in cost from interchanging jobs α and $e_{\min}$. Clearly the completion time of $e_{\min}$ in the new schedule equals that of α in Figure 1, so we only compute Δ with respect to the cost of scheduling the earlier of the two jobs. The relative values of C'_α and d after the interchange provide three subcases (2A, 2B, 2C). In each case, $\Delta < 0$ provides the necessary contradiction.

2A. $C'_\alpha < d$. Then $\Delta = (p_{e_{\min}} - L) - (p_\alpha - L) = p_{e_{\min}} - p_\alpha < 0$.

2B. $C'_\alpha = d$. Then $\Delta = 0 - (p_\alpha - L) = L - p_\alpha < 0$.

2C. $C'_\alpha > d$. Then $\Delta = (L - p_{e_{\min}}) - (p_\alpha - L) = 2L - p_\alpha - p_{e_{\min}} \leq -p_{e_{\min}} < 0$.

Consider a schedule in Case 3. Let Δ denote the change in cost from interchanging jobs α and $t_{\min}$. Again, we need compute only the cost of scheduling the earlier of the two jobs. There are three subcases (3A, 3B, 3C).

3A. $C'_{t_{\min}} > d$. Then $\Delta = [p_{t_{\min}} - (p_\alpha - L)] - L = p_{t_{\min}} - p_\alpha < 0$.

3B. $C'_{t_{\min}} = d$. Then $\Delta = 0 - L < 0$.

3C. $C'_{t_{\min}} < d$. Then $\Delta = [(p_\alpha - L) - p_{t_{\min}}] - L = p_\alpha - p_{t_{\min}} - 2L < 0$.

This completes the proof of Property 3. Thus, we know that there exists an optimal schedule with $p_\alpha \leq p_{e_{\min}}$, or $p_\alpha \leq p_{t_{\min}}$, or both.

Remark 1. It should be easy to see that Property 3 holds if the objective function is

$$z(\sigma) = \sum_{j=1}^n |C_j - d|^r \quad \text{for } r \geq 1.$$

The following definitions for weakly V-shaped schedules will be useful for our dynamic programming algorithm.

Definition 3. A schedule is **early V-shaped (EVS)** if $p_\alpha \leq p_{\min}$.

Definition 4. A schedule is **tardy V-shaped (TVS)** if $p_\alpha \leq p_{\min}$.

In searching for an optimal schedule, then, it is enough from Property 3 to examine all EVS and TVS schedules.

In Properties 4–6, we examine relationships between E , E' and T , with regard to the number of jobs and the total processing time in those sets. Let σ^* denote an optimal schedule, where s is the first job scheduled, and α is defined as before. This schedule is illustrated in Figure 2. By Property 1, we assume without loss of generality that $t^* = 0$ or $C_\alpha = d$.

In the proofs of Properties 4–6, we let Δ denote the change in cost resulting from various possible adjustments to σ^* . Clearly, $\Delta \geq 0$, or the assumption that σ^* is optimal is contradicted. The properties derived here differ from those in Part I due to the possibility that no job finishes at d .

Property 4. If $t^* > 0$, then $|E| \geq |T| - 1$.

Proof. By our above assumption, we must have $C_\alpha = d$ because by hypothesis $t^* > 0$. Start all jobs earlier by $0 < \epsilon < 1$ units of time. Then

$$\begin{aligned} \Delta &= \epsilon[|E| + 1 - |T|] \geq 0 \\ \Rightarrow |E| &\geq |T| - 1. \end{aligned}$$

Property 5. $|E| \leq |T| + 1$.

Proof. Start all jobs later by $0 < \epsilon < 1$ units of time. Then

$$\begin{aligned} \Delta &\leq \epsilon[|T| + 1 - |E|] \geq 0 \\ \Rightarrow |E| &\leq |T| + 1. \end{aligned}$$



Figure 2. Schedule for optimality conditions.

Property 6. If $C_\alpha = d$, then $\sum_{j \in E'} p_j \leq \sum_{j \in T} p_j + 2p_s$, where job s is the first job scheduled.

Proof. Move job s to the last position in the schedule, following T . Then

$$\begin{aligned} \Delta &= \sum_{j \in T} p_j + p_s - \left(\sum_{j \in E'} p_j - p_s \right) \geq 0 \\ \Rightarrow \sum_{j \in E'} p_j &\leq \sum_{j \in T} p_j + 2p_s. \end{aligned}$$

2. NP-COMPLETENESS

The recognition version of CET is: Does there exist a nonpreemptive, single machine schedule satisfying $C_j \geq p_j$, $j = 1, \dots, n$, and having total earliness and tardiness no larger than some integer y ? In Part I, the related problem WET is shown to be NP-complete by providing a reduction from the even-odd partition problem which is known (Garey, Tarjan and Wilfong 1988) to be NP-complete in the ordinary sense. The latter problem can be used with CET.

In particular, we prove that answering the even-odd partition problem, with data $a_1 < \dots < a_{2n}$, is equivalent to answering the recognition question for CET for the instance P1 which has $4n + 3$ jobs defined as follows.

Instance P1

$$p_{2i-1} = a_{2i-1} + B^i, \quad i = 1, \dots, n,$$

$$p_{2i} = a_{2i} + B^i, \quad i = 1, \dots, n,$$

$$p_{2n+1} = B^{n+1},$$

$$p_{2n+i} = 2d, \quad i = 2, 3, \dots, 2n + 3,$$

$$d = B + \sum_{i=1}^{n+1} B^i$$

and

$$y = \sum_{i=1}^n (n - i + 1)(p_{2i-1} + p_{2i})$$

$$+ (2n + 2) \sum_{i=1}^{2n+1} p_i + (2n + 2)^2 d$$

where $B = \sum_{i=1}^{2n} a_i/2$ and B^i denotes B raised to the power i . Observe that for this definition $p_1 \leq \dots \leq p_{4n+3}$, provided that $B \geq 3$ and integer. We make these assumptions without loss of generality.

Lemma 1. If there exists an even-odd partition, then there exists a schedule σ for P1 with $z(\sigma) = y$.

Proof. By assumption, an even–odd partition exists. Define b_i , $i = 1, \dots, 2n$, so that $\{b_{2i-1}, b_{2i}\} = \{a_{2i-1}, a_{2i}\}$ and $\sum_{i=1}^n b_{2i-1} = \sum_{i=1}^n b_{2i}$. The schedule σ is illustrated in Figure 3. Now

$$\begin{aligned} z(\sigma) &= 0(p_{2n+1}) + 1(p_{2n} + p_{2n-1}) + \dots + n(p_2 + p_1) \\ &\quad + \left(\sum_{i=1}^{2n+1} p_i - d + 2d \right) + \left(\sum_{i=1}^{2n+1} p_i - d + 4d \right) \\ &\quad + \dots + \left[\sum_{i=1}^{2n+1} p_i - d + (4n + 4)d \right] = y. \end{aligned}$$

Lemma 2. $z(\sigma) \leq y$ for an optimal schedule σ only if jobs $2n + 2, 2n + 3, \dots, 4n + 3$ are processed consecutively at the end of a schedule which starts at time 0.

Proof. From Properties 2 and 3, any optimal schedule, σ , with $t^* = 0$ in which jobs $2n + 2, 2n + 3, \dots, 4n + 3$ are not scheduled last must process exactly one such job before jobs $1, \dots, 2n + 1$. Then,

$$\begin{aligned} z(\sigma) &= (4n + 3)d + (2n + 1)(2n + 2)d \\ &\quad + (2n + 2) \sum_{i=1}^{2n+1} p_i + \sum_{i=1}^{2n} (2n + 1 - i)p_i, \end{aligned}$$

for such a schedule σ . Thus, the difference

$$\begin{aligned} z(\sigma) - y &= (2n + 1)d \\ &\quad + \sum_{i=1}^n [(n - i + 1)p_{2i-1} + (n - i)p_{2i}] \end{aligned}$$

is positive. Finally, $|T| \geq 2n + 2$ and $|E| \leq 2n$ for each schedule in which jobs $2n + 2, 2n + 3, \dots, 4n + 3$ are scheduled last and $t^* > 0$. Thus, by Property 4, no such schedule can be optimal.

From Lemma 2, we know that $t^* = 0$, and that jobs $2n + 2, 2n + 3, \dots, 4n + 3$ are processed last. The cost of those jobs can be shown, as in the proof of Lemma 1, to equal the last two terms of y . We henceforth consider only the cost of the scheduling jobs $1, \dots, 2n + 1$ consecutively, starting at time 0.

Lemma 3. $z(\sigma) \leq y$ for an optimal schedule σ only if $|E| = n$, $C_\alpha = d$ and job $2n + 1$ is scheduled first in σ .

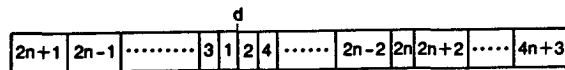


Figure 3. Schedule for Lemma 1.

Proof. The first term in the cost of y , i.e., $0(p_{2n+1}) + 1(p_{2n} + p_{2n-1}) + \dots + n(p_2 + p_1)$, equals the optimal cost of scheduling jobs $1, \dots, 2n + 1$ in an unconstrained scheduling problem.

Since both $|E'| = |T'| + 1$ and the longest job is scheduled first in any optimal schedule for the unconstrained scheduling problem with an odd number of jobs, we have $C_\alpha = d$ in σ , and σ begins with job $2n + 1$.

From Lemma 3, we know that the first $n + 1$ jobs scheduled must complete processing exactly at d .

Lemma 4. $z(\sigma) \leq y$ for an optimal schedule σ only if $\sum_{j \in E'}^{2n} a_j = \sum_{j \in T'}^{2n} a_j$, and exactly one of jobs $2i - 1$ and $2i$ is in E' (and the other is in T'), $i = 1, \dots, n$.

Proof. From Lemma 3, job $2n + 1$ must be scheduled first, then $C_{2n+1} = B^{n+1}$. Moreover, $C_\alpha = d$. Then, since the first $n + 1$ jobs complete processing exactly at time d ,

$$\sum_{j \in E'}^{2n} p_j = B + \sum_{i=1}^n B' = \sum_{j \in T'}^{2n} p_j.$$

From the definitions in P1, this is possible only if E' (and thus T') contains exactly one of jobs $2i - 1$ and $2i$, $i = 1, \dots, n$. Thus

$$\sum_{j \in E'}^{2n} a_j = B = \sum_{j \in T'}^{2n} a_j,$$

and an even–odd partition exists.

Theorem 1. The recognition version of problem CET is NP-complete in the ordinary sense.

Proof. It is easy to see that the recognition version of CET is in NP, and that it takes only polynomial time to construct P1 from an instance of the even–odd partition problem. The equivalence of the two problems follows from Lemma 1 (the if part) and Lemmas 2–4 (the only if part).

As a result of this proof of NP-completeness, we do not expect to find a polynomial algorithm for CET. Subsequent to the submission of our paper, an alternative proof of Theorem 1 was derived by Hoogeveen and van de Velde (1989).

3. A DYNAMIC PROGRAMMING ALGORITHM

Part I describes a highly efficient dynamic programming algorithm that solves to optimality large

instances of WET. A more complex algorithm can be used effectively for CET. The amount of time needed by this algorithm is pseudopolynomial. It follows that CET is not NP-complete in the strong sense. Consequently, Theorem 1 establishes a precise definition of the complexity of CET.

Before describing our algorithm (Optcet) formally, we need to provide some insight into its operation. We partition the solution space into schedules with $t^* = 0$, and those with $t^* > 0$. Clearly if $t^* = 0$, all jobs are scheduled in the time interval $[0, P]$, where $P = \sum_{j=1}^n p_j$. From Property 3, any optimal schedule is either EVS or TVS. It will be clear that Subroutine EVS (respectively, TVS) will find an optimal EVS (respectively, TVS) schedule which completes at time P . Alternatively, suppose that $t^* > 0$. Then, from Property 1, we can assume that $C_a = d$. It will also be clear that Subroutine Nosplit will find an optimal schedule in which a job completes at d . Note that Subroutine Nosplit is essentially the dynamic programming algorithm of Part I, without the optimality conditions derived from the existence of weights in WET. The lowest cost schedule offered by the three Subroutines EVS, TVS and Nosplit is an optimal schedule. All three subroutines make use of the V-shaped structure identified in Properties 2 and 3. It is sufficient to consider only schedules with integer completion times. This will obviously occur if $t^* = 0$, processing is consecutive and processing times are integer. Similarly, it will occur if d is integer and $C_a = d$. From Property 1, one or other of these situations will hold.

Subroutines EVS and TVS consider the jobs in the order $n, n-1, \dots, 1$. From Property 3, job n either starts at 0 or ends at P . We keep track of the total processing time of previously scheduled jobs which finish before d in EVS, and after d in TVS. Subroutine Nosplit considers the jobs in the order $1, 2, \dots, n$, and the state variable is the same as in Subroutine EVS.

Furthermore, since Kanet provides an efficient algorithm for the unconstrained version of this problem, we recommend trying one of the $2^{\lfloor n/2 \rfloor}$ schedules (see Hall 1986), which that algorithm can provide. Among those schedules, the one which has the smallest total processing time before d will schedule jobs $(n, n-2, \dots, n-2\lfloor(n-2)/2\rfloor)$ early. If the total processing time of those jobs is no greater than d , an optimal schedule for the unconstrained problem is feasible, and thus optimal, for CET. Finally, an optimal schedule is easily found if d is so small that, whatever schedule is used, no job is early (see Bagchi, Sullivan and Chang 1986).

Algorithm Optcet

Step 0. Given $0 < p_1 \leq \dots \leq p_n$ and $d > 0$.

Step 1. If $p_n + p_{n-2} + \dots + p_{n-2\lfloor(n-2)/2\rfloor} \leq d$, construct an optimal schedule

$$\sigma^* = (n, n-2, \dots, n-2\lfloor(n-2)/2\rfloor, \dots, n-3, n-1).$$

Then $z(\sigma^*) = \sum_{j=1}^n \lfloor j/2 \rfloor p_{n-j+1}$. Stop.

If $d \leq p_1$, construct an optimal SPT schedule, $\sigma^* = (1@p_1, 2, \dots, n)$. Then $z(\sigma^*) = \sum_{j=1}^n (n-j+1)p_j - nd$. Stop.

Otherwise, go to Step 2.

Step 2. Call EVS.

Step 3. Call TVS.

Step 4. Call Nosplit.

Step 5. Find the optimal solution value, from $\min\{z(\sigma_{\text{EVS}}^*), z(\sigma_{\text{TVS}}^*), z(\sigma_{\text{Nosplit}}^*)\}$. The corresponding schedule is optimal. Stop.

Subroutine EVS

Let $f_k(a)$ = the minimum cost to schedule jobs $n, n-1, \dots, n-k+1$ provided that the latest job scheduled which finishes at or before d , finishes at time $d-a$, $a \geq 0$, and the earliest job scheduled which finishes after d starts at time $\sum_{i=1}^{n-k} p_i - a + d$. (That is, the latest such job finishes at P .)

Recurrence relation:

$$\begin{aligned} f_{k+1}(a) &= \min \left\{ a + f_k(a + p_{n-k}), \sum_{i=1}^{n-k} p_i - a + f_k(a) \right\} \\ &\quad \text{if } \sum_{i=1}^{n-k} p_i > a \quad \text{and } a + p_{n-k} \leq d. \\ &= a + f_k(a + p_{n-k}) \\ &\quad \text{if } \sum_{i=1}^{n-k} p_i \leq a \quad \text{and } a + p_{n-k} \leq d. \\ &= +\infty \quad \text{otherwise.} \end{aligned}$$

Boundary condition:

$$\begin{aligned} f_0(a) &= 0 \quad \text{for } a = d \\ &= +\infty \quad \text{for } a \neq d. \end{aligned}$$

Minimum cost schedule defined by:

$$z(\sigma_{\text{EVS}}^*) = \min_{0 \leq a \leq d} f_n(a).$$

Complexity: $O(nd)$, since $k \leq n$, $a \leq d$ by assumption, and we can compute $\sum_{i=1}^{n-k} p_i$, $k = 1, \dots, n-1$ in $O(n)$ time before using the recurrence relation.

Subroutine TVS

Let $g_k(m)$ = the minimum cost to schedule jobs $n, n-1, \dots, n-k+1$, provided that the earliest job scheduled which starts at or after d , starts at time $d+m$, $m \geq 0$, and the latest job scheduled which starts before d finishes at time $d+m - \sum_{i=1}^{n-k} p_i$. (That is, the earliest such job starts at 0.)

Recurrence relation:

$$\begin{aligned}
 g_{k+1}(m) &= \min \left\{ \left| \sum_{i=1}^{n-k-1} p_i - m \right| + g_k(m), \right. \\
 &\quad \left. m + p_{n-k} + g_k(m + p_{n-k}) \right\} \\
 &\quad \text{if } \sum_{i=1}^{n-k} p_i > m \text{ and } m + p_{n-k} \leq P - d. \\
 &= m + p_{n-k} + g_k(m + p_{n-k}) \\
 &\quad \text{if } \sum_{i=1}^{n-k} p_i \leq m \text{ and } m + p_{n-k} \leq P - d. \\
 &= +\infty \text{ otherwise.}
 \end{aligned}$$

Boundary condition:

$$\begin{aligned}
 g_0(m) &= 0 \quad \text{for } m = P - d \\
 &= +\infty \quad \text{for } m \neq P - d.
 \end{aligned}$$

Minimum cost schedule defined by:

$$z(\sigma_{\text{TVS}}^*) = \min_{0 \leq m \leq P-d} g_n(m).$$

Complexity:

$$O[n(P-d)], \text{ since } k \leq n, \quad m \leq P-d.$$

Subroutine Nosplit

Let $h_k(e)$ = the minimum cost to schedule jobs $1, 2, \dots, k$, without the due date splitting any job, given that the total processing time of jobs scheduled early (or on time) is e .

Recurrence relation:

$$\begin{aligned}
 h_{k+1}(e) &= \min \left\{ e - p_{k+1} + h_k(e - p_{k+1}), \sum_{i=1}^{k+1} p_i - e + h_k(e) \right\} \\
 &\quad \text{if } e \geq p_{k+1} \\
 &= \sum_{i=1}^{k+1} p_i - e + h_k(e) \text{ otherwise.}
 \end{aligned}$$

Boundary condition:

$$\begin{aligned}
 h_0(e) &= 0 \quad \text{if } e = 0 \\
 &= +\infty \quad \text{if } e \neq 0.
 \end{aligned}$$

Minimum cost schedule defined by:

$$z(\sigma_{\text{Nosplit}}^*) = \min_{0 \leq e \leq d} h_n(e).$$

Complexity:

$$O(nd), \text{ since } k \leq n, \quad e \leq d.$$

Remark 2. In view of Subroutines EVS and TVS, it is sufficient to consider the values of e from 0 to $d-1$ in Nosplit.

Two points about the subroutines are worthy of note. In each case, the first alternative in the recurrence relation represents the cost of scheduling the next job as early as possible, and the second alternative, similarly, as late as possible. In subroutine TVS, scheduling a job as early as possible may still mean that it finishes after d , hence the need for the absolute value sign ($| \cdot |$).

Computational Improvements

We are able to reduce the amount of enumeration needed through the use of simple properties, and the optimality conditions of Section 1. For Subroutines EVS, TVS and Nosplit, the recurrence relation at stage k is computed over the values $s^k = s_{\min}^k, s_{\min}^k + 1, \dots, s_{\max}^k$ for the state variable s^k , where s^k denotes a^k, m^k , or e^k , respectively. Recall that in EVS and TVS, we schedule jobs beginning from the outside of the interval $[0, P]$. A lower bound on a^k (respectively, m^k) is provided by an expression involving the total processing time of previously scheduled jobs. Similarly, in Nosplit, e^k is bounded above first by the total processing time of previously scheduled jobs, second by a bound derived from Property 6, and finally, by $d-1$ from Remark 2. The exact computation performed is defined by:

EVS

$$a_{\min}^k = \max \left\{ 0, d - \sum_{j=n-k+1} p_j \right\}, \quad a_{\max}^k = d.$$

TVS

$$m_{\min}^k = \max \left\{ 0, P - d - \sum_{j=n-k+1} p_j \right\}, \quad m_{\max}^k = P - d.$$

Nosplit

$$e_{\min}^k = 0,$$

$$e_{\max}^k = \min \left\{ \sum_{j=1}^k p_j, \left\lfloor \frac{\sum_{j=1}^n p_j}{2} \right\rfloor + p_n, d - 1 \right\}.$$

Our results suggest that the use of $a_{\min}^k$ and $m_{\min}^k$ reduces computational effort in EVS and TVS by approximately 20% in those subroutines, and the combined effect of the bounds on $e_{\max}^k$ in Nosplit is a 40% reduction. We now provide a summary of a numerical example of the algorithm.

Consider the instance [6; 1, 3, 7, 8, 9; 14]. It is easy to check that neither of the conditions of Step 1 of Optcet is met. The three subroutines provide schedules which are summarized and illustrated below. The numbers inside the figures (see Figures 4–6) represent job processing times. Recall that both EVS and TVS schedule jobs in decreasing index order, but that Nosplit schedules jobs in increasing index order.

EVS

$$\begin{aligned} f_0(14) = 0 &\rightarrow f_1(5) = 5 \rightarrow f_2(5) = 27 \\ &\rightarrow f_3(5) = 41 \rightarrow f_4(5) = 47 \\ &\rightarrow f_5(2) = 49 \rightarrow f_6(1) = 50. \end{aligned}$$

Thus $\sigma_{\text{EVS}}^* = (6, 2, 1, 3@20, 4, 5)$, and $z(\sigma_{\text{EVS}}^*) = 50$.

TVS

$$\begin{aligned} g_0(22) = 0 &\rightarrow g_1(13) = 22 \rightarrow g_2(5) = 35 \\ &\rightarrow g_3(5) = 41 \rightarrow g_4(5) = 42 \\ &\rightarrow g_5(2) = 47 \rightarrow g_6(1) = 49. \end{aligned}$$

Thus $\sigma_{\text{TVS}}^* = (4, 3@15, 1, 2, 5, 6)$, and $z(\sigma_{\text{TVS}}^*) = 49$.

Nosplit

$$\begin{aligned} h_0(0) = 0 &\rightarrow h_1(1) = 0 \rightarrow h_2(4) = 1 \\ &\rightarrow h_3(4) = 8 \rightarrow h_4(4) = 23 \\ &\rightarrow h_5(4) = 46 \rightarrow h_6(13) = 50. \end{aligned}$$

Thus $\sigma_{\text{Nosplit}}^* = (6, 2, 1@, 3, 4, 5)$, and $z(\sigma_{\text{Nosplit}}^*) = 50$.

By comparing the values of the three schedules thus identified, we conclude that the optimal schedule is the one found by TVS, with a cost of 49. We note that the scheduling of jobs in the same sequence in

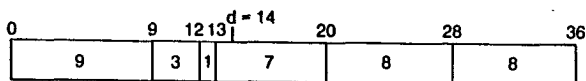


Figure 4. Best schedule from EVS.

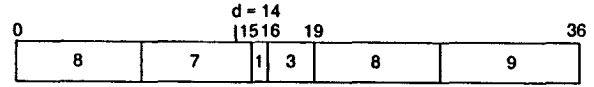


Figure 5. Best schedule from TVS.

Subroutines EVS and Nosplit is a coincidence, and will not generally occur. For the instance [3; 5, 10, 20; 10], the best EVS schedule is (1, 2@15, 3), but the best Nosplit schedule is (2@, 1, 3).

Theorem 2. *Algorithm Optcet finds an optimal schedule for every instance of CET.*

Proof. From Property 1, we need only show that Nosplit examines all potentially optimal schedules in which $t^* > 0$, and that EVS and TVS together examine all potentially optimal schedules in which $t^* = 0$. It is easy to see that Nosplit could allow any job to finish either early (including on time) or late, while maintaining Property 2. Thus, any optimal schedule with $t^* > 0$ is examined by Nosplit.

Now, neither EVS nor TVS individually examines all potentially optimal schedules in which $t^* = 0$. However, it is easy to see that each does so until it schedules job α . Let $\sigma^* = (\dots, e_{\min}^*, \alpha^*, t_{\min}^*, \dots)$ denote an optimal schedule with $t^* = 0$. EVS can only miss this schedule if $t_{\min}^*$ is not scheduled by the time α^* is scheduled. For example, consider the instance [2; 1, 11; 10] for which the optimal schedule of (2@11, 1) has a cost of 3. EVS will set $C_2 = 12$, and the resulting schedule is (1, 2@12), with a cost of 11. Since $t_{\min}^*$ is not scheduled at the time α^* is scheduled, we know that $p_{\alpha^*} \geq p_{t_{\min}^*}$. However, if $p_{\alpha^*} = p_{t_{\min}^*}$, the jobs are interchangeable and the EVS schedule is optimal. So, if EVS finds a nonoptimal schedule, then $p_{\alpha^*} > p_{t_{\min}^*}$. A similar argument shows that if TVS finds a nonoptimal schedule, then $p_{\alpha^*} > p_{e_{\min}^*}$. Thus, if both EVS and TVS find nonoptimal schedules, it follows that $p_{\alpha^*} > \max\{p_{e_{\min}^*}, p_{t_{\min}^*}\}$. This implies that σ^* is not weakly V-shaped. But we know from Property 3 that any optimal schedule is weakly V-shaped. This contradicts the assumption that neither EVS nor TVS finds an optimal schedule.

The overall complexity of Optcet is $O(n \sum_{j=1}^n p_j)$. Thus, Optcet is linear with respect to a unary encoding of the data, or is pseudopolynomial. We note that the worst case complexity is the same as that of Optwet in Part I, however, the average running time is somewhat greater because three subroutines are used instead of one.

In order to test the computational effectiveness of Algorithm Optcet, the algorithm is coded in FORTRAN on a Prime 9955 minicomputer, and

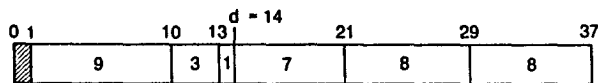


Figure 6. Best schedule from Nosplit.

compiled using the F77 compiler. We randomly generate 240 test problems, consisting of 24 series of 10 problems each. The number of jobs (n) in a problem has values of 100, 200, 500 and 1,000. Processing times are independently generated from a uniform distribution over the integers $1, \dots, \text{TOPSIZ}$, where TOPSIZ has values of 10, 20, 50 and 100. The due date, d , is given a value of $\lfloor t \sum_{j=1}^n p_j \rfloor$, where t is a parameter specifying the restrictiveness of the due date constraint. That parameter is allowed to have values of 0.2 and 0.4.

The results in Table I show the mean and the maximum solution times, over the 10 random problems in each series. The times shown are in Prime 9955 CPU seconds, and exclude input and output time. Computation times increase approximately linearly with TOPSIZ, and in proportion to n^2 , but are not much influenced by the value of t . The results confirm that our implementation of Optcet routinely solves to optimality large random instances of CET, within three minutes of minicomputer time. Our study increases the size of instance which is known to

Table I
Computation Times for Random Problems

Series	n	TOPSIZ	t	Mean CPU Time (sec.)	Maximum CPU Time (sec.)
1	100	10	0.2	0.87	0.99
2	100	10	0.4	0.90	0.98
3	100	20	0.2	1.73	1.99
4	100	20	0.4	1.71	1.95
5	100	50	0.2	4.85	5.14
6	100	50	0.4	4.34	4.58
7	100	100	0.2	9.22	10.07
8	100	100	0.4	8.79	9.83
9	200	10	0.2	3.58	3.87
10	200	10	0.4	3.42	3.65
11	200	20	0.2	6.99	7.32
12	200	20	0.4	6.82	7.27
13	200	50	0.2	17.98	19.65
14	200	50	0.4	17.06	18.05
15	500	10	0.2	23.04	25.57
16	500	10	0.4	21.99	25.45
17	500	20	0.2	43.72	49.17
18	500	20	0.4	43.92	48.00
19	500	50	0.2	111.33	119.27
20	500	50	0.4	107.04	118.88
21	1,000	10	0.2	93.32	102.88
22	1,000	10	0.4	90.10	96.30
23	1,000	20	0.2	169.91	178.77
24	1,000	20	0.4	165.05	171.53

be solvable from $n = 25$ (Szwarc) to $n = 1,000$. Such improvement is made possible by the new optimality conditions that we provide, and which enable us to prove the optimality of our dynamic programming algorithm.

Remark 3. We also note that Remark 1 in Section 1 makes it possible to use the EVS and TVS subroutines of Optcet for any $r > 1$. Furthermore, it is possible to modify the Nosplit subroutine to address the resulting nonlinear problem, provided that job splitting is constrained to be at integer times. This modification is necessary, because for $r > 1$, Property 1 does not hold, i.e., an instance with all optimal solutions having $t^* > 0$ and $C_\alpha > d$ cannot be ruled out.

4. EFFICIENTLY SOLVABLE CASE

In this section, we consider a special case of CET for which an optimal solution can be found in polynomial time. In this special case, jobs are either “small” or “large,” and all the small jobs are scheduled before all the large jobs. A more precise statement follows. Its proof is based on the proof of Theorem 1.

Theorem 3. If there exists $k \geq \lceil n/2 \rceil$ satisfying $\sum_{j=1}^k p_j < d$ and $p_{k+1}, \dots, p_n > 2d$, then an easily found schedule σ^* with $C_\alpha = d$, $|E'| = \lceil n/2 \rceil$, and $\{k+1, \dots, n\} \in T$ is optimal.

Proof. Let σ denote the schedule $(k, k-1, \dots, 1 @, k+1, \dots, n)$, where $z(\sigma) = 0p_k + 1p_{k-1} + \dots + (k-1)p_1 + 1p_n + 2p_{n-1} + \dots + (n-k)p_{k+1}$. From Properties 1 and 2, any schedule σ' beginning with job $k+i$, where $1 \leq i \leq n-k$, has $t^* = 0$ and

$$\begin{aligned} z(\sigma') &= n(p_{k+i} - d) + (n-1)p_1 + (n-2)p_2 + \dots \\ &\quad + (n-k)p_k + \dots + (n-k-i+1)p_{k+i-1} \\ &\quad + (n-k-i)p_{k+i+1} + \dots + 2p_{n-1} + 1p_n. \end{aligned}$$

Then

$$\begin{aligned} z(\sigma') - z(\sigma) &= (n-k) \sum_{j=1}^k p_j - \sum_{j=k+1}^{k+i-1} p_j + n(p_{k+i} - d) \\ &\geq np_{k+i} - nd - (n-k)p_{k+i} \\ &= kp_{k+i} - nd > 2kd - nd \geq 0. \end{aligned}$$

While schedule σ is not necessarily optimal, it is lower in cost than any schedule beginning with a job $k+i$, $1 \leq i \leq n-k$. Thus, no optimal schedule can begin with such a job. Furthermore, by Properties 2 and 3, jobs $k+1, \dots, n$ are scheduled last. Since $k \geq \lceil n/2 \rceil$

and $\sum_{j=1}^k p_j < d$, it follows that $t^* > 0$, and then from Property 1 we may assume that $C_\alpha = d$. Now using Properties 4 and 5, we know there exists an optimal schedule with $|E'| = \lceil n/2 \rceil$. This schedule can easily be found in $O(n \log n)$ time using positional penalties $0, 1, \dots, \lceil n/2 \rceil - 1$ for the early jobs and $n - \lceil n/2 \rceil, n - \lceil n/2 \rceil - 1, \dots, n - k + 1$ for the late jobs from $1, \dots, k$. The largest job has the smallest positional penalty, and so on. Note that the positional penalties corresponding to jobs $k + 1, \dots, n$ are now fixed because those jobs are known to appear at the end of the schedule. If the jobs arrive ordered by processing time, the solution can be found only $O(n)$ time.

5. SUMMARY AND CONCLUSIONS

The earliness and tardiness problem is one of the most natural and important problems in machine scheduling with nonregular performance measures. We have shown that the version with an arbitrary, and thus potentially constraining, due date is a hard problem in a complexity sense. In doing so, we confirm a conjecture by Baker and Scudder. The complexity of many nonregular measure scheduling problems is now defined because they are generalizations either of this problem, or of the unconstrained problem with weights studied in Part I. Examples include problems where the weight of a job depends on whether it is early or late, problems with more than one due date, and problems with release times.

The design of a pseudopolynomial algorithm, moreover, shows that the earliness and tardiness problem is NP-complete in the ordinary, rather than the strong, sense. Despite our complexity result, we have shown that the problem is computationally tractable. Indeed, our algorithm increases the size of instance which is solvable from 25 to 1,000 jobs. We also identify a special case of the problem which can be solved by a polynomial algorithm.

Several closely related problems remain as topics for future investigation. Among these are problems where more than one machine is available. While there is a variety of possible configurations for such machines, we conjecture, based on our results here, that most of those problems are NP-complete. Possible exceptions include extreme special cases such as unit processing times. A relevant topic that interests the authors is machine scheduling where the due date to which a job is assigned can, at least to some extent, be chosen (Panwalkar, Smith and Seidmann 1982). Finally, the design and analysis of heuristics for hard problems of this type would be an innovative and valuable contribution. We believe that the close relationship between current issues in inventory control

and the class of earliness and tardiness problems will motivate increasing research interest in the future.

ACKNOWLEDGMENT

Two referees provided comments which improved the presentation of the paper. This research was supported in part by NSERC grants A4619, OGP0105675 and in part by the Manufacturing Research Corporation of Ontario.

REFERENCES

- BAGCHI, U., R. S. SULLIVAN AND Y. L. CHANG. 1986. Minimizing Mean Absolute Deviation of Completion Times About a Common Due Date. *Naval Res. Logist. Quart.* 33, 227-240.
- BAGCHI, U., R. S. SULLIVAN AND Y. L. CHANG. 1987. Minimizing Mean Squared Deviation of Completion Times About a Common Due Date. *Mgmt. Sci.* 33, 894-906.
- BAKER, K. R., AND G. D. SCUDDER. 1990. Sequencing With Earliness and Tardiness Penalties: A Review. *Opns. Res.* 38, 22-36.
- EILON, S., AND I. G. CHOWDHURY. 1977. Minimizing Waiting Time Variance in the Single Machine Problem. *Mgmt. Sci.* 23, 567-575.
- GAREY, M. R., R. E. TARJAN AND G. T. WILFONG. 1988. One-Processor Scheduling With Earliness and Tardiness Penalties. *Math. Opns. Res.* 13, 330-348.
- HALL, N. G. 1986. Single and Multiple Processor Models for Minimizing Completion Time Variance. *Naval Res. Logist. Quart.* 33, 49-54.
- HALL, N. G., AND M. E. POSNER. 1991. Earliness-Tardiness Scheduling Problems, I: Weighted Deviation of Completion Times About a Common Due Date. *Opns. Res.* 39, 836-846.
- HOOGVEEN, J. A., AND S. L. VAN DE VELDE. 1989. Scheduling Around a Small Common Due Date. Manuscript, Centre for Mathematics and Computer Science, Amsterdam, The Netherlands.
- KANET, J. J. 1981. Minimizing the Average Deviation of Job Completion Times About a Common Due Date. *Naval Res. Logist. Quart.* 28, 643-651.
- PANWALKAR, S., M. SMITH AND A. SEIDMANN. 1982. Common Due Date Assignment to Minimize Total Penalty for the One Machine Scheduling Problem. *Opns. Res.* 30, 391-399.
- RAGHAVACHARI, M. 1986. A V-Shape Property of Optimal Schedule of Jobs About a Common Due Date. *Eur. J. Opnl. Res.* 23, 401-402.
- SZWARC, W. 1989. Single Machine Scheduling to Minimize Absolute Deviation of Completion Times From a Common Due Date. *Naval Res. Logist.* 5, 663-674.
- VANI, V., AND M. RAGHAVACHARI. 1987. Deterministic and Random Single Machine Sequencing With Variance Minimization. *Opns. Res.* 35, 111-120.